

AI BLOG-NEST API

Project Report — Phases 1 to 6

An AI-Powered Blog Content Management System

Team ID: SWTID-2026-4186

Team Leader : JAISIN S

Email ID : jaisinshanmugam2004@gmail.com

Team member : GUNANITHI A

Email ID : gunagunanithi31@gmail.com

Team member : GOKUL K

Email ID : gokulgokul57069@gmail.com

Team member : VEERAPPAN C

Email ID : veerappanc64@gmail.com

Table of Contents

- Phase 1 — Brainstorming & Ideation
- Phase 2 — Requirement Analysis
- Phase 3 — Project Design
- Phase 4 — Project Planning
- Phase 5 — Project Development
- Phase 6 — Project Testing

Phase 1 — Brainstorming & Ideation

Project Title

AI BlogNest — AI-Powered Blog Content Management System

1. Introduction

- AI BlogNest is an AI-powered backend REST API designed to simplify blog creation, content management and AI-assisted writing.
- The system allows users to create and manage blog posts, organize content using categories and tags, manage comments, and use Artificial Intelligence to generate blog content and summarize existing content.
- The project uses Node.js and Express.js for backend development, MongoDB with Mongoose for database management, and Google Gemini AI for AI-based content generation.

2. Problem Identification

Traditional blog management can become difficult when users have to manage a large number of posts, drafts, categories, tags and comments manually. The identified problems are:

- Difficulty in managing multiple blog drafts.
- Difficulty organizing categories and tags.
- Time-consuming blog content creation.
- Difficulty in managing comments and moderation.
- Lack of role-based access for different types of users.
- Difficulty in handling content approval and publishing.
- Need for AI assistance in creating and summarizing content.
- Need for a secure and centralized backend system.

The project document specifically identifies difficulties in managing drafts, categories, tags, post performance, content approval, comments, media and user permissions.

3. Proposed Solution

To solve these problems, we propose AI BlogNest, a centralized backend API that combines normal blog management features with Artificial Intelligence. The system provides:

- User registration and login.
- Secure authentication using JWT.
- Role-based access control.
- Blog creation and management.
- Categories and tags.
- Comment management and moderation.
- Blog search.
- AI-powered blog generation.
- AI-powered blog summarization.
- AI FAQ generation.
- AI-based WeatherWise and FitTrack assistance.
- Secure database management using MongoDB.
- Input validation, sanitization and rate limiting.

The project document describes the backend as a modern AI-powered Content Management System combining traditional blogging capabilities with Generative AI.

4. Purpose of the Project

The main purpose of AI BlogNest is to provide a secure and intelligent platform for creating, managing and organizing blog content while reducing the time and effort required for content creation through Artificial Intelligence. The project aims to:

- Simplify blog creation and management.
- Reduce the time required to write blog content.
- Provide AI assistance for generating blog content.
- Provide AI-based summarization of long blog content.
- Organize blogs using categories and tags.
- Provide controlled access based on user roles.
- Improve comment management and moderation.
- Provide a secure backend API for future frontend applications.
- Maintain blog and user data efficiently using MongoDB.
- Provide a scalable architecture for future enhancements.

5. Target Users

Role	Responsibilities
Admin	Manages users, roles and permissions, administrative activities, and inappropriate comments.
Editor	Reviews blog content, manages publishing workflows, content moderation, categories and editorial activities.
Author	Creates blog posts, manages their own content, organizes drafts and blog information.
Reader	Reads published blogs, searches for content, adds comments.

These four roles are defined in the project documentation.

6. Brainstorming / Ideation

During brainstorming, the following ideas were considered for improving the traditional blog management process:

Idea	Purpose
Blog Management	Create, update, view and delete blogs
AI Blog Generation	Generate blog content from a topic
AI Summarization	Create short summaries from long content
Categories & Tags	Organize and search blog content
Comment Management	Allow and moderate user comments
Role-Based Access	Provide different permissions to users
Search	Find blogs based on relevant content
Media Management	Maintain information about media associated with blogs
Security	Protect APIs and user information

7. Innovation / Uniqueness of the Project

The main idea behind AI BlogNest is to combine a Blog Content Management System with Generative AI. Instead of only storing and displaying blog posts, the system can assist users in the content creation process. The AI functionality includes:

- AI blog generation.
- AI content summarization.
- AI FAQ assistance.
- AI WeatherWise insights.
- AI FitTrack recommendations.

This makes the system more than a basic blogging API and provides intelligent assistance for content-related activities.

8. Expected Benefits

- Faster content creation.
- Easier blog management.
- Better content organization.
- Reduced manual effort.
- Secure user authentication.
- Controlled access based on roles.
- Easier comment moderation.
- AI assistance for content generation.
- Quick summarization of lengthy content.
- Centralized data management.
- Reusable backend APIs for web or mobile applications.

9. Expected Outcome

At the end of the project, AI BlogNest will provide a functional backend REST API through which users can register, log in, create and manage blogs, interact with comments, and access AI-powered content features. The project will also provide a foundation that can later be connected to a frontend web or mobile application.

Phase 2 — Requirement Analysis

Requirement analysis explains what is needed to develop the AI BlogNest system. The project uses Node.js, Express.js, MongoDB, and Google Gemini AI.

2. Functional Requirements

The system should:

- Allow users to register and login.
- Provide different roles: Admin, Editor, Author, and Reader.
- Allow users to create, view, update, and delete blogs.
- Allow users to like and search blogs.
- Support categories and tags.
- Allow users to add comments.
- Allow admins and editors to manage comments.
- Generate blogs using Google Gemini AI.
- Summarize blogs using AI.
- Provide secure login using JWT.

3. Non-Functional Requirements

The system should be:

- Secure
- Fast
- Reliable
- Easy to use
- Easy to maintain
- Scalable
- Cross-platform

4. Software Requirements

Software	Purpose
Windows / macOS / Linux	Operating System
Node.js	Backend
Express.js	API
MongoDB	Database
Mongoose	Database management
Google Gemini AI	AI features
Postman	API testing
Visual Studio Code	Development
GitHub	Code storage

5. Hardware Requirements

Hardware	Requirement
Processor	Intel i5 / Ryzen 5 or above
RAM	8 GB minimum
Storage	1 GB free space

6. Database Requirements

The system uses MongoDB. Main data stored:

- User
- Blog
- Comment

7. API Requirements

The system provides APIs for:

- Registration
- Login
- Blog management
- Comments
- Blog search
- AI blog generation
- AI summarization

8. Authentication

The system uses JWT authentication.

Register → Login → Verify User → Generate JWT → Access System

9. User Roles

Role	Work
Admin	Manage the system
Editor	Manage and review blogs
Author	Create blogs
Reader	Read blogs and comment

10. Summary

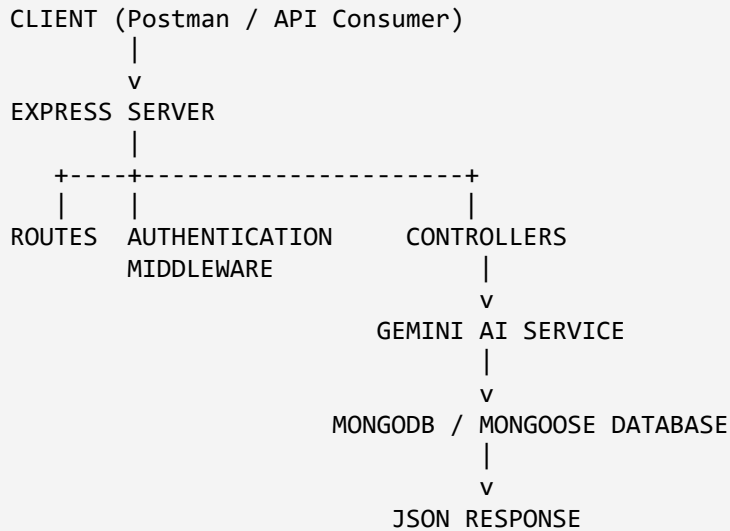
AI BlogNest requires:

- Node.js
- Express.js
- MongoDB
- Mongoose
- Google Gemini AI
- JWT
- bcrypt.js
- Blog management
- Comment management
- Search
- Security
- API testing tools

Phase 3 — Project Design

Architecture is based on a layered/MVC-style backend architecture. The project contains models, controllers/routes and services, with MongoDB handled through Mongoose.

Architecture



Database Design

User

- `_id`
- `name`
- `email`
- `password`
- `role`

Blog

- `_id`
- `userID`
- `name/title`
- `content`

Phase 4 — Project Planning

This phase explains how we planned to complete the project.

Week Work Plan

Day	Activities
Day 1	Brainstorming, Requirement Analysis & Project Planning — finalize project idea, problem statement, objectives, functional/non-functional requirements, user roles, and technology stack.
Day 2	Project Setup & Architecture — set up Node.js, Express.js, project structure, MVC architecture, environment configuration, and MongoDB connection.
Day 3	MongoDB, Mongoose & Authentication — create User model, configure Mongoose, implement user registration, password hashing, login, and JWT authentication.
Day 4	Authorization & Blog CRUD APIs — implement role-based authorization and create, read, update, and delete blog APIs.
Day 5	Gemini AI Integration — integrate Google Gemini API and implement AI blog generation and content summarization.
Day 6	Testing & Bug Fixing — test registration, login, JWT, authorization, blog CRUD, AI generation, and summarization using Postman/Thunder Client; fix errors.
Day 7	Documentation & Demonstration — prepare project report, screenshots, GitHub repository, presentation, demo script, and final project demonstration.

Phase 5 — Project Development

Module 1: Project Setup

```
src/
├── models/
├── controllers/
├── routes/
├── services/
├── middleware/
└── config/
```

Module 2: Core API Endpoints

1. User Registration

METHOD: POST <http://localhost:5000/api/auth/register>

```
{
  "name": "ishamalini",
  "email": "ishamalini67567@gmail.com",
  "password": "123456"
}
```

2. User Login

METHOD: POST <http://localhost:5000/api/auth/login>

```
{
  "email": "ishamalini67567@gmail.com",
  "password": "123456"
}
```

3. Create Blog

METHOD: POST <http://localhost:5000/api/blogs>

```
{
  "title": "Introduction to Generative AI",
  "content": "Generative AI is a technology that can create new content using artificial intelligence.",
  "category": "Artificial Intelligence"
}
```

4. Fetch Blog

METHOD: GET <http://localhost:5000/api/blogs>

5. Generate Blog

METHOD: POST <http://localhost:5000/api/ai/generate-blog>

```
{  
  "topic": "Generative AI"  
}
```

6. Summarize Blog

METHOD: POST <http://localhost:5000/api/ai/summarize>

```
{  
  "content": "Generative AI is a branch of artificial intelligence that can generate new  
content such as text, images, and code."  
}
```

Phase 6 — Project Testing

The following test cases validate the core API endpoints of AI BlogNest.

1. User Registration

API: POST `http://localhost:5000/api/auth/register`

Expected Result: 201 Created

```
{
  "message": "User registered successfully",
  "user": {
    "id": "[user_id]",
    "name": "ishamalini",
    "email": "[your email]"
  }
}
```

2. User Login

API: POST `http://localhost:5000/api/auth/login`

Expected Result: JWT token

```
{
  "token": "[JWT TOKEN]"
}
```

3. Create Blog

API: POST `http://localhost:5000/api/blogs`

Expected Result: 201 Created

```
{
  "title": "Introduction to Generative AI",
  "content": "Generative AI is a technology that can create new content such as text, images, and code.",
  "category": "Artificial Intelligence",
  "author": "[USER_ID]",
  "authorName": "ishamalini",
  "_id": "[BLOG_ID]",
  "createdAt": "[DATE_TIME]",
  "updatedAt": "[DATE_TIME]",
  "__v": 0
}
```

4. Fetch Blog

API: GET `http://localhost:5000/api/blogs`

Expected Result: Blog list

5. Generate Blog

API: POST <http://localhost:5000/api/ai/generate-blog>

Expected Result: AI-generated blog

6. Summarize Blog

API: POST <http://localhost:5000/api/ai/summarize>

Expected Result: AI summary

```
{  
  "summary": "Generative AI is a type of artificial intelligence that creates new content  
by learning and using patterns found in existing data."  
}
```

Demo Video : https://drive.google.com/file/d/1HC8y_ggC8qR8IU6T7MpPjdptsviQ4kpL/view?usp=sharing

API Testing : https://drive.google.com/file/d/1Bvi9r4robOXJi_SIIbCCEsNH9VZfzSzN/view?usp=sharing

CodeDrive: <https://github.com/jaisin06/AI-BLOGNEST-API>